Électif Technologies Web 2.0 - 2018
TEA "TW2 Suggest"

Objectif : Terminer le cas d'étude suggest en autonomie.

- Nous repartons du code tel qu'il était à la fin de la séance précédente. Ce sujet **remplace** le sujet initial TW2 Suggest (exercices A4 et A5).
- Le tableau ci-dessous présente les différentes versions des codes client et serveur à développer :

Client	Caractéristiques	Avantages Inconvénients	Serveur	Caractéristiques	Avantages Inconvénients
suggest_V1.html	La saisie d'un texte déclenche une requête asynchrone vers la page fixture La réponse du serveur est affichée dans le cadre des suggestions Un clic sur une suggestion remplace le contenu du champ d'entrée texte		fixture.php	Renvoie toujours les mêmes données, structurées en XHTML Attend deux secondes avant de répondre	Permet de développer rapidement la couche d'intégration Permet de valider l'UX en cas de fortes latences du serveur
suggest_V2.html	La requête asynchrone est envoyée vers la page data.php On envoie des données à la page data.php avec la requête AJAX (debutNom=<chaine saisie>)		exo 1 data.php	Renvoie les prénoms et les noms correspondant aux données reçues, lus depuis un fichier Utilise des expressions régulières	La structure renvoyée : Prénom Nom rend le client dépendant du serveur La structure ne permet pas de distinguer le prénom du nom
exo 2 suggest_V3.html	On récupère du JSON, il faut le parcourir pour intégrer les suggestions dans la page	On insère dans le champ le contenu intégral de la suggestion cliquée	exo 2 data_json.php	Renvoie les prénoms et noms correspondant aux données reçues, au format JSON, lus depuis un fichier	JSON est structuré JSON est moins verbeux que XML JSON est disponible dans tous les langages de programmation, ce qui rend le client indépendant du serveur
suggest_V4.html	La page envoie une requête vers data_bdd.php		exo 3 data_bdd.php	Renvoie les prénoms et noms correspondant aux données reçues, au format JSON, lus depuis une base de données	
exo4 suggest_V5.html	On met en place un cache dans le navigateur qui stocke les méta-données des suggestions actuellement affichées Les suggestions et le cadre de saisie affichent des choses différentes	On peut distinguer le visuel des données	idem		
exo5 suggest_V6.html	On met en place un cache dans le navigateur qui stocke les réponses du serveur au fur et à mesure On ne redemande pas au serveur des données qu'il nous a déjà envoyées	On améliore les performances en termes de consommation de bande passante réseau	idem		

Travail demandé :

Exercice 1 (data.php)

La recherche de correspondance doit être effectuée sur les noms et les prénoms ; la page doit afficher prénom et nom dans cet ordre.

Ainsi, la saisie de la chaîne “BE” devrait renvoyer :

ELIAN BERNARD
MARCEAU BEYOU
BENJAMIN CHAPERON
BENJAMIN SONNEVILLE

Nous souhaitons que les noms et prénoms affichés dans les suggestions soient typographiés en utilisant une majuscule en début de Nom/Prénom et des minuscules ensuite.

Exercice 2 (suggest_V3.html et data_json.php)

La structure de données renvoyée par la page data_json.php doit être du JSON, qui doit contenir les champs ‘nom’ et ‘prénom’ pour chaque étudiant.
La page doit renvoyer un tableau d’étudiants dans tous les cas, même si aucun étudiant n’est trouvé.

La page suggest_V3.html doit interpréter le flux JSON reçu pour transformer la chaîne de caractères renvoyée en un objet.

La page doit vérifier la présence d’un étudiant dans le tableau, et parcourir les étudiants pour les afficher (au format “<initiale du prénom> <8 premiers caractères du Nom>”) dans la liste des suggestions.

Exercice 3 (suggest_V4.html et data_bdd.php)

A l’aide du fichier sql fourni, créer une base de données contenant les étudiants du module TW2.

Utiliser l’opérateur LIKE pour rechercher des étudiants correspondant aux mêmes critères que précédemment dans la base de données (la librairie maLibSQL.pdo.php est fournie dans l’archive moodle)

Le flux JSON renvoyé par le serveur est généré à l’aide des méta-données fournies par la fonction parcoursRs(), auxquelles on ajoutera un groupe TP, calculé de telle sorte que les étudiants dont le nom est situé avant Cyprien Hue (exclu) appartiennent au groupe 1, et les autres au groupe 2.

Exercice 4 (suggest_V5.html)

Nous souhaitons que le clic sur une suggestion n’insère pas exactement le contenu de la suggestion dans le champ d’entrée texte.
Les suggestions porteront l’information “<initiale du prénom> <8 premiers caractères du Nom>”, mais le cadre de saisie devra contenir <prénom> <initiale du nom> (<indice de l’étudiant en base>)

Pour cela, vous enregistrerez l’indice de l’étudiant en base comme attribut “id” de chaque suggestion, et vous stockerez dans un cache côté client (i.e. une variable globale) la liste des étudiants actuellement affichés dans les suggestions. Lors du clic sur un étudiant, vous récupérerez l’indice de cet étudiant dans le cache, ce qui vous permettra de consulter le cache pour remplir le champ de saisie au format demandé.

Comment afficher le contenu du cache dans la console de développement du navigateur ?

Exercice 5 (suggest_V6.html)

Nous souhaitons éviter de faire deux fois une requête vers le serveur avec la même chaîne de caractères. Pour cela, après chaque nouvelle requête envoyée, on stockera dans un cache les résultats obtenus, en les associant à la chaîne envoyée au serveur.

Lorsque l'utilisateur saisira une nouvelle chaîne, on vérifiera que les étudiants concernés ne sont déjà en cache. Si c'est le cas, on affichera les suggestions à partir du cache, sans faire de requête au serveur.